
КАФЕДРА

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

должность, уч. степень, звание

подпись, дата

инициалы, фамилия

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ
Нечеткая логика: подбор команды исполнителей
по курсу: Методы исследований в менеджменте

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

подпись, дата

инициалы, фамилия

Санкт-Петербург 2025

Цель работы:

Научиться оценивать качество соискателей на должность, формируя количественную оценку годности кандидата.

Задание:

Решить задачу подбора команды исполнителей для ситуации, связанной с выполнением одной из стратегий технологического предпринимательства.

Выполнение задания:

1. Фаззификация: каждое входное число превращается в степени принадлежности $\mu \in [0,1]$ к термам (например, возраст=35 даёт какие-то μ к «молодой/средний/выше_среднего» — `фп_возраст.png`).
2. Оценка правил: для каждого правила берётся $AND = \min(\dots\mu\dots)$ по всем условиям (`age & edu & exp & soft`). Это «вклад» правила.
3. Агрегация по выводам: все правила, ведущие к одному выводу (например, «высокое»), объединяются $OR = \max$.
4. Дефаззификация: итоговое нечеткое множество по «качество» превращается в одно число через центроид (центр тяжести). Это и есть балл (0..100), который мы пишем в таблицу.

результаты_100_кандидатов.tsv — все 100 сгенерированных кандидатов, отсортированы по качеству (0–100) по убыванию. Колонки: ID, возраст, образование (1–10), опыт (лет), софт (1–10), итоговый балл.

```
ID Возраст Образование(1-10) Опыт(лет) Софт(1-10) Качество(0-100)
38 40 9 5 1 86.7
3 39 2 5 9 86.6
44 34 5 5 9 86.6
72 42 8 5 7 86.2
78 38 10 5 1 86.2
27 36 9 5 3 85.2
30 37 10 5 4 85.2
6 38 7 5 5 84.4
7 36 7 5 7 84.4
```

команда_порог_70.tsv — подмножество из предыдущего файла с баллом ≥ 70 (по методичке это «очень нужные»). Это и есть наша подобранная команда.

```
ID Возраст Образование(1-10) Опыт(лет) Софт(1-10) Качество(0-100)
38 40 9 5 1 86.7
3 39 2 5 9 86.6
44 34 5 5 9 86.6
72 42 8 5 7 86.2
78 38 10 5 1 86.2
27 36 9 5 3 85.2
30 37 10 5 4 85.2
6 38 7 5 5 84.4
7 36 7 5 7 84.4
42 43 4 4 9 84.4
67 36 9 4 5 84.4
82 39 10 2 9 84.4
83 41 7 1 10 84.4
84 42 9 2 10 84.4
41 31 9 4 7 82.9
87 45 7 5 5 82.4
93 28 10 4 2 70.1
```

инфо_система_НЛ.txt — краткая спецификация FIS: тип Мамдани, дефаззификация (centroid), число правил, диапазоны и термы переменных.

Тип: Мамдани, дефаззификация = центроид (centroid)

Число правил: 24

Входы: возраст[20..45], образование[1..10], опыт[1..5], софт[1..10];

Выход: качество[0..100]

правила_HЛ.txt — все 24 правила в явном виде: строки вида
ЕСЛИ age=... И edu=... И exp=... И soft=... ТО quality=....
Это аналог Rule Editor в MATLAB.

Коротко как это работает:

1. В функции build_fis() я перечисляю термы для каждого входа:

age_terms = ['молодой','средний','выше_среднего'], edu_terms =
['бакалавр','магистр'], exp_terms = ['малый','большой'], soft_terms =
['хорошо','отлично'].

2. Далее идут 4 вложенных цикла ($3 \times 2 \times 2 \times 2 = 24$ комбинации). Для каждой комбинации считается число «сильных» признаков:

- возраст $\in \{\text{средний, выше_среднего}\} \rightarrow +1$
- образование = магистр $\rightarrow +1$
- опыт = большой $\rightarrow +1$
- софт = отлично $\rightarrow +1$

3. По этому счётчику задаётся заключение (выход quality):

- strong $\geq 3 \rightarrow$ высокое
- strong = 2 $\rightarrow$ среднее
- иначе $\rightarrow$ малое

Гаусс, трапеция, треугольник

фп_возраст.png — три gaussmf: «молодой», «средний», «выше_среднего». По оси X — возраст, по Y — степень принадлежности $\mu \in [0,1]$.

фп_образование.png — две trapmf: «бакалавр», «магистр». Покрывают шкалу 1–10 с нахлёстом

фп_опыт.png — две trimf: «малый» и «большой» опыт на шкале 1–5.

фп_софт.png — две trapmf: «хорошо», «отлично», полностью покрывают 1–10

фп_качество.png — выходные trimf: «малое», «среднее», «высокое» на шкале 0–100.

просмотр_возраст_топ.png, ...образование_топ.png, ...опыт_топ.png, ...софт_топ.png — для лучшего (топ-1) кандидата: вертикальная линия показывает его чёткое значение; закрашка/подсветка — какие термы активировались и насколько.

просмотр_качество_топ.png — агрегированное выходное нечёткое множество (после правил) + вертикальная линия на дефазифицированном значении (итоговый балл).

- Возраст (age) — гауссовы (gaussmf), 3 шт.

Возраст — «плавная» величина без жёстких границ. Колоколообразные кривые дают мягкие переходы между «молодой → средний → выше среднего», без резких скачков и с естественным перекрытием.

- Образование (edu) — трапецидальные (trapmf), 2 шт. («бакалавр», «магистр»)

По сути, две категории. Трапеция даёт «плато» уверенной принадлежности (например, 8–10 — точно «магистр») и пологие склоны для пограничных значений (7–8) — удобно моделировать нечёткость границы.

- Опыт (exp) — треугольные (trimf), 2 шт. («малый», «большой»)

Две режимные зоны. Треугольник — минималистичная форма (мало параметров), даёт линейный, легко интерпретируемый рост принадлежности от «малого» к «большому» через центр 3 года.

- Софт-навыки (soft) — трапецидальные (trapmf), 2 шт. («хорошо», «отлично»)

Оценки по шкале 1–10 часто «ступенчатые»: есть зона уверенно «хорошо» и зона уверенно «отлично». Трапеции обеспечивают полное покрытие диапазона 1...10 без «дыр» (важно для стабильного вывода) и мягкие границы между уровнями.

- Качество (quality) — треугольные (trimf), 3 шт. («малое», «среднее», «высокое»)

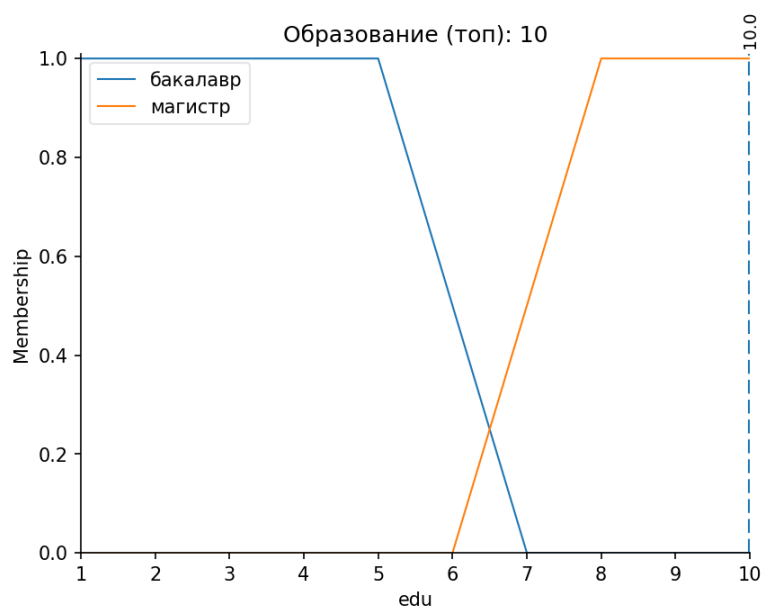
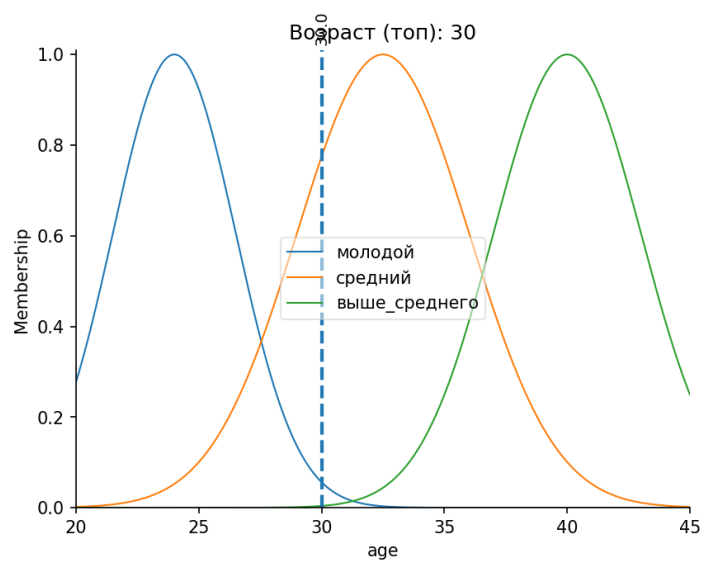
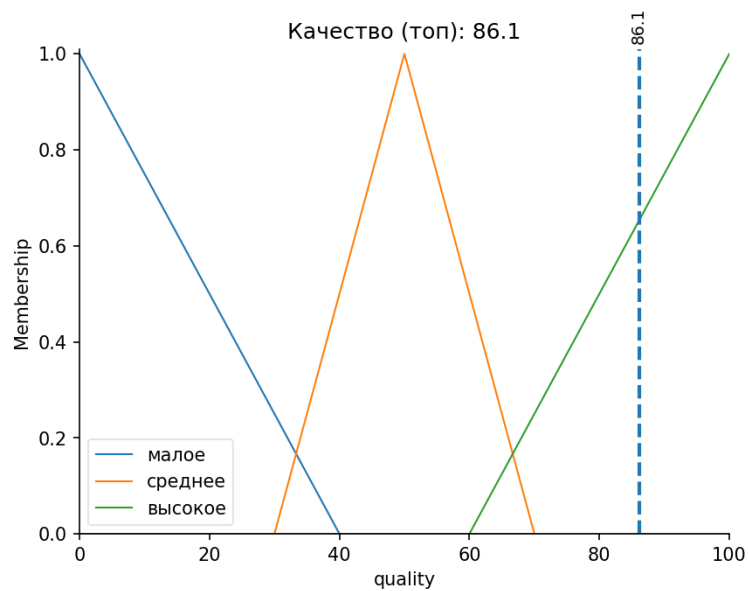
Выход — дискретные уровни пригодности. Треугольники дают читаемые «корзины» с перекрытием; центры на $\sim 0/50/100$ упрощают интерпретацию и дефаззификацию.

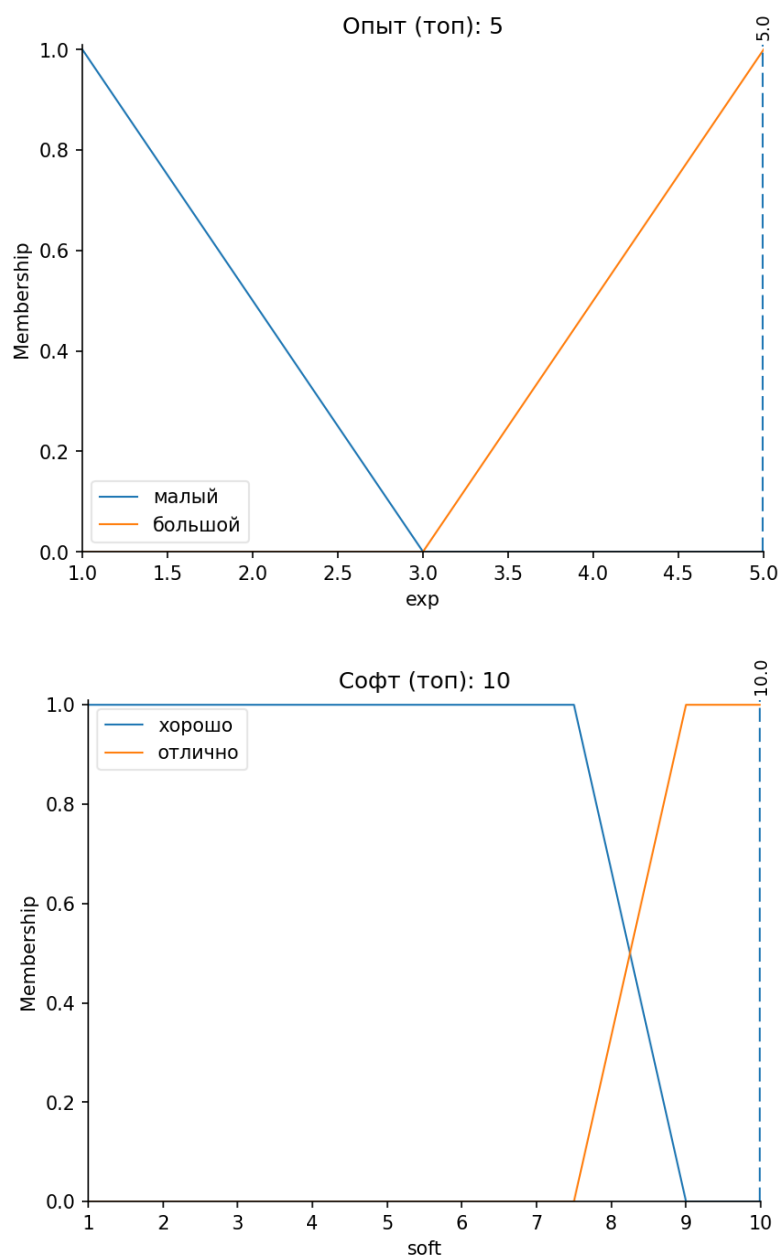
- Дефаззификация — centroid (центроид)

Стандарт для Мамдани: устойчив, учитывает всю площадь агрегированного множества (не только максимум), даёт «средневзвешенное» решение — хорошо для ранжирования кандидатов.

Общие соображения:

- У всех входов кривые перекрываются, чтобы не было «чёрных дыр» (иначе выход может стать неопределённым).
- Выбраны формы с простой интерпретацией и малым числом параметров (легко объяснить на защите и подстроить при необходимости).





Вывод:

Реализована система нечёткого вывода Мамдани для оценки кандидатов по четырём критериям с заданными функциями принадлежности и 24 правилами; сгенерировано 100 заявок, выполнен ранжирующий отбор по порогу ≥ 70 баллов, результаты и графики подтверждают корректность фаззификации/дефаззификации. Модель воспроизводит экспертную логику (образование, опыт и софт-навыки повышают «качество») и может быть оперативно перенастроена изменением ФП и правил под конкретную роль.

Листинг:

```
import os
import csv
import numpy as np

import matplotlib
matplotlib.use("Agg") # сохраняем в файлы, без окон
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = 'DejaVu Sans' # кириллица в подписях

import skfuzzy as fuzz
from skfuzzy import control as ctrl

# --- Папка для всех результатов ---
OUT_DIR = "ЛР3_результаты"

# ----- 1) СБОРКА FIS (Мамдани) -----

def build_fis():
    # Универсы значений
    age = ctrl.Antecedent(np.linspace(20, 45, 501), 'age')
    edu = ctrl.Antecedent(np.linspace(1, 10, 451), 'edu')
    exp = ctrl.Antecedent(np.linspace(1, 5, 401), 'exp')
    soft = ctrl.Antecedent(np.linspace(1, 10, 451), 'soft')
    quality = ctrl.Consequent(np.linspace(0, 100, 1001), 'quality', defuzzify_method='centroid')

    # Функции принадлежности
    age['молодой'] = fuzz.gaussmf(age.universe, 24.0, 2.5) # mean, sigma
    age['средний'] = fuzz.gaussmf(age.universe, 32.5, 3.5)
    age['выше_среднего'] = fuzz.gaussmf(age.universe, 40.0, 3.0)

    edu['бакалавр'] = fuzz.trapmf(edu.universe, [1, 1, 5, 7])
    edu['магистр'] = fuzz.trapmf(edu.universe, [6, 8, 10, 10])

    exp['малый'] = fuzz.trimf(exp.universe, [1, 1, 3])
    exp['большой'] = fuzz.trimf(exp.universe, [3, 5, 5])

    # Полное покрытие шкалы 1..10 без «дыр»
    soft['хорошо'] = fuzz.trapmf(soft.universe, [1, 1, 7.5, 9])
    soft['отлично'] = fuzz.trapmf(soft.universe, [7.5, 9, 10, 10])

    quality['малое'] = fuzz.trimf(quality.universe, [0, 0, 40])
    quality['среднее'] = fuzz.trimf(quality.universe, [30, 50, 70])
    quality['высокое'] = fuzz.trimf(quality.universe, [60, 100, 100])

    # Правила (24) через «подсчёт сильных условий»
    age_terms = ['молодой', 'средний', 'выше_среднего']
    edu_terms = ['бакалавр', 'магистр']
    exp_terms = ['малый', 'большой']
    soft_terms = ['хорошо', 'отлично']

    rules = []
    text_rules = []
```

```

for a in age_terms:
    for e in edu_terms:
        for x in exp_terms:
            for s in soft_terms:
                strong = 0
                if a in ('средний', 'выше_среднего'): strong += 1
                if e == 'магистр': strong += 1
                if x == 'большой': strong += 1
                if s == 'отлично': strong += 1

                if strong >= 3:
                    cons = 'высокое'
                elif strong == 2:
                    cons = 'среднее'
                else:
                    cons = 'малое'

                antecedent = (age[a] & edu[e] & exp[x] & soft[s])
                rules.append(ctrl.Rule(antecedent, quality[cons]))
                text_rules.append(f"ЕСЛИ age={a} И edu={e} И exp={x} И soft={s} ТО quality={cons}")

system = ctrl.ControlSystem(rules)
return system, (age, edu, exp, soft, quality), rules, text_rules

# ----- 2) УТИЛИТЫ -----

def p(fn): # путь в папку результатов
    return os.path.join(OUT_DIR, fn)

def save_tsv(path, header, rows):
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f, delimiter="\t")
        if header: w.writerow(header)
        w.writerows(rows)

def eval_candidate(system, a, e, x, s):
    sim = ctrl.ControlSystemSimulation(system, flush_after_run=1)
    sim.input['age'] = float(a)
    sim.input['edu'] = float(e)
    sim.input['exp'] = float(x)
    sim.input['soft'] = float(s)
    sim.compute()
    out = sim.output.get('quality', None)
    if out is None or np.isnan(out):
        return 0.0
    return float(out)

def save_view_marked(var, filepath, crisp=None, sim=None, title=None):
    """
    Рисует график ФП как обычно, + вертикальная пунктирная линия на crisp-значении.
    Для выхода можно передать crisp = sim.output['quality'].
    """
    plt.ioff()

```

```

var.view(sim=sim)          # базовый график из skfuzzy
ax = plt.gca()
if crisp is not None:
    ax.axvline(float(crisp), linestyle='--', linewidth=2)
    ax.text(float(crisp), 1.02, f"{float(crisp):.1f}",
            rotation=90, va='bottom', ha='center', fontsize=9)
if title:
    ax.set_title(title)
fig = plt.gcf()
fig.savefig(filepath, dpi=150, bbox_inches='tight')
plt.close(fig)

def print_table(rows, header, title=None, max_rows=10):
    if title: print(f"\n{title}")
    show = rows[:max_rows]
    widths = [len(h) for h in header]
    for r in show:
        for i, cell in enumerate(r):
            widths[i] = max(widths[i], len(str(cell)))
    def line(L=' ', M='+', R=' ', fill='-'):
        return L + M.join(fill*(w+2) for w in widths) + R
    print(line())
    print(" | " + " | ".join(str(h).ljust(widths[i]) for i, h in enumerate(header)) + " | ")
    print(line(' |', '+', '|'))
    for r in show:
        print(" | " + " | ".join(str(r[i]).ljust(widths[i]) for i in range(len(header))) + " | ")
    if len(rows) > max_rows:
        print(" | " + " | ".join("..." for i in range(len(header))) + " | ")
    print(line(' |', '+', '|'))

# ----- 3) ОСНОВНОЙ СЦЕНАРИЙ -----

def main():
    os.makedirs(OUT_DIR, exist_ok=True)

    system, (age, edu, exp, soft, quality), rules, text_rules = build_fis()

    # Инфо + правила
    with open(p("инфо_система_НЛ.txt"), "w", encoding="utf-8") as f:
        f.write("Тип: Мамдани, дефаззификация = центроид (centroid)\n")
        f.write(f"Число правил: {len(rules)}\n")
        f.write(f"Входы: возраст[20..45], образование[1..10], опыт[1..5], софт[1..10]; Выход: качество[0..100]\n")
    with open(p("правила_НЛ.txt"), "w", encoding="utf-8") as f:
        for i, r in enumerate(text_rules, 1):
            f.write(f"{i:02d}. {r}\n")

    # ФП (как в редакторе FIS)
    save_view_marked(age, p("фп_возраст.png"))
    save_view_marked(edu, p("фп_образование.png"))
    save_view_marked(exp, p("фп_опыт.png"))
    save_view_marked(soft, p("фп_софт.png"))
    save_view_marked(quality, p("фп_качество.png"))

```

```

# --- Генерация ровно 100 кандидатов (полностью случайно, без фиксированного seed) ---
rng = np.random.default_rng() # каждый запуск — новые данные
N = 100
ages = rng.integers(20, 46, size=N)
edus = rng.integers(1, 11, size=N)
exps = rng.integers(1, 6, size=N)
softs = rng.integers(1, 11, size=N)

batch = []
for i in range(N):
    q = eval_candidate(system, ages[i], edus[i], exps[i], softs[i])
    batch.append([i+1, int(ages[i]), int(edus[i]), int(exps[i]), int(softs[i]), f"{q:.1f}"])

# Сортировка по качеству
batch_sorted = sorted(batch, key=lambda r: float(r[-1]), reverse=True)
save_tsv(p("результаты_100_кандидатов.tsv"),
        ["ID", "Возраст", "Образование(1-10)", "Опыт(лет)", "Софт(1-10)", "Качество(0-100)"],
        batch_sorted)

print_table(batch_sorted[:10],
        ["ID", "Возраст", "Образование(1-10)", "Опыт(лет)", "Софт(1-10)", "Качество(0-100)"],
        title="Топ-10 из 100 (по качеству)")

# Порог для команды
THRESH = 70.0
team = [r for r in batch_sorted if float(r[-1]) >= THRESH]
save_tsv(p("команда_порог_70.tsv"),
        ["ID", "Возраст", "Образование(1-10)", "Опыт(лет)", "Софт(1-10)", "Качество(0-100)"],
        team)
print(f"\nПорог команды: ≥ {THRESH:.0f} баллов — выбрано {len(team)} из {N} кандидатов.")

# Просмотр переменных/выхода для Топ-1 кандидата (как Variable Viewer)
if batch_sorted:
    top = batch_sorted[0] # [ID, age, edu, exp, soft, quality_str]

    sim = ctrl.ControlSystemSimulation(system, flush_after_run=1)
    sim.input['age'] = float(top[1])
    sim.input['edu'] = float(top[2])
    sim.input['exp'] = float(top[3])
    sim.input['soft'] = float(top[4])
    sim.compute()
    q_crisp = float(sim.output['quality'])

    save_view_marked(age, p("просмотр_возраст_топ.png"),
        crisp=top[1], sim=sim, title=f"Возраст (топ): {top[1]}")
    save_view_marked(edu, p("просмотр_образование_топ.png"),
        crisp=top[2], sim=sim, title=f"Образование (топ): {top[2]}")
    save_view_marked(exp, p("просмотр_опыт_топ.png"),
        crisp=top[3], sim=sim, title=f"Опыт (топ): {top[3]}")
    save_view_marked(soft, p("просмотр_софт_топ.png"),
        crisp=top[4], sim=sim, title=f"Софт (топ): {top[4]}")
    save_view_marked(quality, p("просмотр_качество_топ.png"),
        crisp=q_crisp, sim=sim, title=f"Качество (топ): {q_crisp:.1f}")

```

```

# Просмотр по одному правилу для каждого класса вывода
idx_low = idx_mid = idx_high = None
for i, txt in enumerate(text_rules):
    if ("quality=малое" in txt) and (idx_low is None): idx_low = i
    if ("quality=среднее" in txt) and (idx_mid is None): idx_mid = i
    if ("quality=высокое" in txt) and (idx_high is None): idx_high = i
    if idx_low is not None and idx_mid is not None and idx_high is not None:
        break
if idx_low is not None:
    rules[idx_low].view(); plt.gcf().savefig(p("правило_малое.png"), dpi=150, bbox_inches='tight'); plt.close()
if idx_mid is not None:
    rules[idx_mid].view(); plt.gcf().savefig(p("правило_среднее.png"), dpi=150, bbox_inches='tight'); plt.close()
if idx_high is not None:
    rules[idx_high].view(); plt.gcf().savefig(p("правило_высокое.png"), dpi=150, bbox_inches='tight'); plt.close()

# Итог: перечислим файлы
print("\nФайлы сохранены в папку:", os.path.abspath(OUT_DIR))

if __name__ == "__main__":
    main()

```